

Gen-AI Architecture

A Complete Visual Guide

LLM | RAG | Vector DB | MCP | Agentic AI | Agents | n8n

LLM	RAG	Vector DB
MCP	Agentic AI	n8n

This document provides a comprehensive overview of the modern Generative AI stack — from foundational large language models to fully autonomous agentic systems. Each concept is explained with diagrams, real-world flow examples, and practical implementation guidance.

1. Large Language Model (LLM)

A **Large Language Model (LLM)** is a neural network trained on vast amounts of text data to understand and generate human language. Based on the Transformer architecture, LLMs learn statistical relationships between tokens and can reason, summarise, translate, and generate content across virtually any domain.

How It Works

Stage	What Happens
1. Pre-training	Model reads billions of documents, learns token probabilities
2. Tokenisation	Text split into tokens (~4 chars each); model thinks in tokens
3. Attention	Transformer weighs relationships between all tokens in context
4. Prediction	Model predicts next token — everything emerges from this
5. Fine-tuning / RLHF	Model tuned on instructions + human feedback to be helpful

Key Concepts

Term	Meaning
Parameters	Neural network weights — more = more capable (8B to 1T+)
Context Window	Max tokens model sees at once (e.g. 200 000 tokens)
Temperature	0 = deterministic output 1+ = creative / random
Token	Chunk of text (~4 chars); models price by token count
Hallucination	Model generates plausible-sounding but incorrect facts
RLHF	Reinforcement Learning from Human Feedback — aligns the model

Popular LLMs

Model	Provider	Notes
Claude Sonnet / Opus	Anthropic	Strong reasoning, long context, safe
GPT-4o	OpenAI	Multimodal, widely integrated
Gemini 1.5 Pro	Google	Very long context window (1M tokens)
Llama 3	Meta	Open-source, self-hostable

2. Retrieval-Augmented Generation (RAG)

RAG enhances LLM responses by first *retrieving* relevant documents from a knowledge base, then feeding those documents as context to the LLM before it generates an answer. This gives the model access to private, up-to-date, or domain-specific knowledge without retraining.

RAG Pipeline — Step by Step

Step	Action	Detail
1	Ingest Documents	Load PDFs, web pages, databases into pipeline
2	Chunk Text	Split into overlapping chunks (~500 tokens each)
3	Embed Chunks	Convert each chunk to a vector using embedding model
4	Store in Vector DB	Save vector + original text in vector database
5	User Query	User asks a question
6	Embed Query	Convert query to a vector (same embedding model)
7	Similarity Search	Find top-k most similar vectors in Vector DB
8	Retrieve Chunks	Fetch the original text of matching chunks
9	Augment Prompt	Inject retrieved chunks into LLM system prompt
10	Generate Answer	LLM answers grounded in the retrieved context

Why RAG?

- Overcomes LLM knowledge cutoff — always uses fresh data
- Enables private knowledge Q&A without exposing data to training
- Reduces hallucinations — LLM cites retrieved source text
- Cheaper than retraining — update the knowledge base, not the model

RAG vs Fine-tuning

	RAG	Fine-tuning
When to use	Dynamic / private data	Fixed domain knowledge
Cost	Low (index update)	High (GPU training)
Freshness	Real-time	Static until re-trained
Transparency	Cites sources	Knowledge baked in

3. Vector Database

A **Vector Database** stores data as high-dimensional numerical vectors (embeddings) and enables ultra-fast *semantic similarity search*. Unlike traditional SQL databases that match exact values, vector DBs find content that is *semantically similar* in meaning — even if the words are different.

Embedding — Text to Numbers

Text: "dog" → [0.21, -0.54, 0.87, 0.33, ...] (1536 dims)
Text: "cat" → [0.19, -0.51, 0.83, 0.31, ...] (similar → nearby)
Text: "car" → [0.92, 0.13, -0.44, 0.71, ...] (different → far away)

Similarity = cosine distance between vectors

SQL vs Vector DB

Feature	SQL Database	Vector Database
Search type	Exact / keyword match	Semantic / meaning match
Query example	WHERE name = 'dog'	find things like dog
Data type	Structured rows	Unstructured text / images
Speed on vectors	Slow (full scan)	Fast (ANN index)
Use case	Transactions, reports	RAG, recommendations, search

Popular Vector Databases

Name	Type	Best For
Pinecone	Managed cloud	Production, zero-ops
Qdrant	Open source	High performance, Rust-based
ChromaDB	Open source	Local dev, Python-native
Weaviate	Open source	Multi-modal + GraphQL API
pgvector	Postgres extension	Teams already using Postgres
FAISS	Library (Meta)	In-memory research / prototyping
Milvus	Open source	Enterprise scale

4. Model Context Protocol (MCP)

MCP is an open standard by Anthropic that provides a unified protocol for AI agents to connect to external tools, APIs, and data sources. Instead of writing custom integration code for every API, developers wrap the API once as an MCP Server — and any MCP-compatible agent can use it immediately.

MCP Architecture

Layer	Component	Responsibility
Agent	LLM + Logic	Decides which tool to call and when
MCP Client	Built into app	Sends tool requests using MCP protocol
MCP Server	API wrapper	Receives MCP calls, executes real API
External API	GitHub / Jira...	The actual service being controlled

MCP Core Concepts

Concept	Description
Tools	Functions the agent can call, e.g. <code>create_issue()</code> , <code>send_message()</code>
Resources	Data sources the agent can read, e.g. files, DB rows
Prompts	Reusable prompt templates exposed by the MCP server
Server	Process that wraps an API and speaks the MCP protocol
Client	Component inside the agent app that connects to servers

Official MCP Servers (ready to use)

Package	Connects To
@modelcontextprotocol/server-github	GitHub — issues, PRs, repos
@modelcontextprotocol/server-slack	Slack — messages, channels
@modelcontextprotocol/server-postgres	PostgreSQL — queries, schema
@modelcontextprotocol/server-file-system	Local file system
@modelcontextprotocol/server-brave-search	Brave web search
@modelcontextprotocol/server-google-maps	Google Maps — directions, places

MCP vs Direct API Integration

	Direct API	MCP
Setup per API	Custom code every time	Standardised once
Reusability	Agent-specific	Any agent can use
Tool discovery	Manual	Automatic
Maintenance	High	Low
Security	Per integration	Centralised

5. Agentic AI

Agentic AI refers to AI systems that autonomously plan, decide, and take sequences of actions to achieve a goal — rather than answering a single prompt. The LLM acts as the reasoning engine inside a loop that continues until the goal is complete.

Traditional LLM vs Agentic AI

Dimension	Traditional LLM	Agentic AI
Input	One prompt	A high-level goal
Output	One response	Series of actions
Memory	None (stateless)	Maintains state across steps
Tools	No	Yes — search, code, APIs
Decision-making	Single step	Multi-step planning & replanning
Loops	No	Runs until goal is achieved

The Agent Loop — ReAct Pattern

Thought : I need to find the current stock price of AAPL.
Action : search('AAPL stock price today')
Observe : AAPL is trading at \$189.43
Thought : I now have the answer.
Answer : Apple (AAPL) is currently trading at \$189.43.

Agent Components

Component	Role
LLM	Reasoning engine — understands goal, plans, decides next action
Memory	Short-term (chat history) + long-term (Vector DB)
Tools	Functions the agent can invoke (search, code, APIs via MCP)
System Prompt	Defines agent persona, constraints, available tools
Orchestrator	Manages the loop, routes to specialist agents, collects results

Multi-Agent Pattern

Complex tasks can be split across **specialist agents** coordinated by an **Orchestrator Agent**:

- **Orchestrator** — receives goal, plans, delegates to specialists
- **Research Agent** — web search, RAG lookups
- **Coder Agent** — writes and executes code
- **Writer Agent** — formats and polishes final output
- **API Agent** — calls external services via MCP

Popular Agent Frameworks

Framework	Language	Notes
Claude Agent SDK	Python	Anthropic-native, MCP-first

Framework	Language	Notes
LangGraph	Python	Graph-based stateful agents
AutoGen	Python	Microsoft, multi-agent conversations
CrewAI	Python	Role-based crew of agents
n8n	Visual	No-code / low-code agent builder

6. n8n — Visual Agentic Workflow Builder

n8n is an open-source workflow automation platform with first-class support for building AI agents visually. It lets you wire together triggers, LLMs, memory, tools, and APIs on a canvas — no boilerplate code needed.

Core Nodes

Node	Purpose
Trigger	Starts the flow — Chat, Webhook, Schedule, Email
AI Agent	The brain: LLM + tools + memory orchestrator
Chat Model	Connect Claude, GPT-4o, Gemini to the agent
Memory	Window Buffer or Vector Store for conversation history
Tool Nodes	SerpAPI, HTTP Request, Gmail, Slack, Jira, DB, etc.
MCP Client Tool	Connect to any MCP server as an agent tool
Execute Workflow	Call a sub-workflow (specialist agent pattern)
Switch / Merge	Route and combine results for multi-agent patterns

AI Agent Node Settings

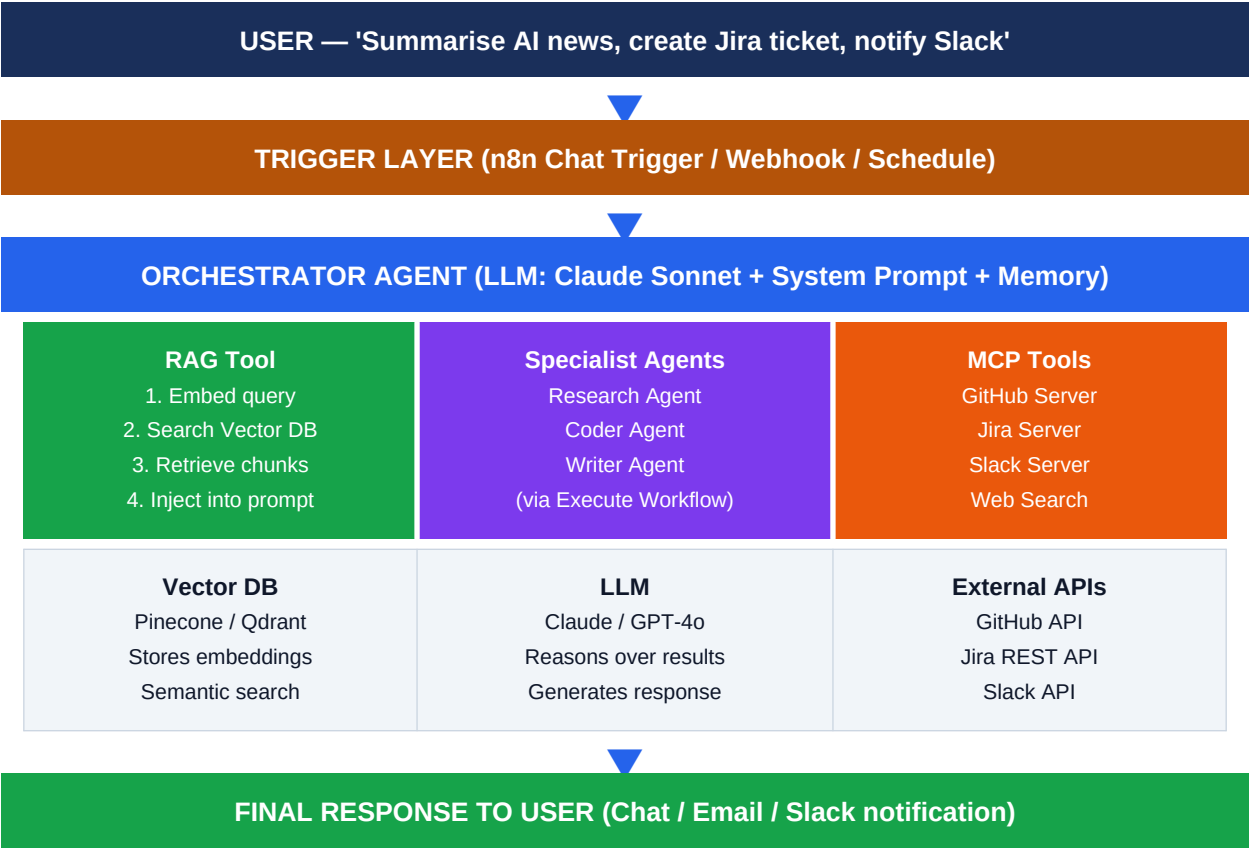
Setting	Recommended Value
System Prompt	Define persona, goal, and available tools
Max Iterations	10–15 (prevents infinite loops)
Agent Type	Tools Agent (most flexible)
Return Intermediate	On during development for debugging

Example Flows in n8n

Flow	Trigger	Agent Tools	Output
Customer Support	Webhook	DB lookup, Shipping API	Email reply
Daily Research Digest	Schedule	Web search, Summarise	Gmail to team
GitHub to Jira Sync	GitHub event	Jira MCP, Slack MCP	Jira ticket + Slack
RAG Q&A Bot	Chat	Vector DB, HTTP Request	Chat response

7. End-to-End Architecture — All Concepts Together

Full System Flow Diagram



Worked Example — Step by Step

#	Who Acts	What Happens
1	User	Asks: 'Summarise today AI news, create Jira ticket, notify Slack'
2	n8n Trigger	Webhook fires, passes message to Orchestrator Agent
3	Orchestrator LLM	Plans 5 steps; chooses web-search tool first
4	MCP: Web Search	Calls Tavily/Brave API, returns 5 articles
5	RAG	Embeds articles, stores in Vector DB for future use
6	Orchestrator LLM	Retrieves chunks, writes 200-word summary
7	MCP: Jira Server	Calls create_issue() → returns ticket DEV-201
8	MCP: Slack Server	Calls send_message(#ai-updates) → team notified
9	Orchestrator LLM	Confirms all steps done, composes final reply
10	User	Receives: summary + 'DEV-201 created + Slack notified'

8. Concept Summary — One-Page Reference

What Each Concept Does

Concept	What It Is	Role in the System
LLM	Neural net trained on text	The brain — understands, reasons, decides
Agentic AI	LLM + loop + tools	Autonomously completes multi-step goals
Agent	LLM + memory + tools	Worker that executes a specific task
RAG	Retrieve docs → feed to LLM	Gives LLM access to private/fresh data
Vector DB	Stores text as vectors	Enables semantic search for RAG
MCP	Standard API protocol	Connects agents to any external API/tool
n8n	Visual workflow builder	Wires all of the above together visually

How They Connect

Every layer depends on the one below. The **LLM** is the foundational engine. **Agentic AI** wraps the LLM in a loop with tools and memory. Individual **Agents** are specialised instances of Agentic AI. **RAG** feeds relevant knowledge to agents using a **Vector DB** as the semantic memory store. **MCP** standardises how agents call external APIs. **n8n** is the canvas that visually connects all layers into production-ready workflows.

Technology Stack at a Glance

Layer	Open-source Options	Managed / Cloud Options
LLM	Llama 3, Mistral, Falcon	Claude, GPT-4o, Gemini
Agent Framework	LangGraph, CrewAI, AutoGen	Claude Agent SDK
RAG Orchestration	LlamaIndex, LangChain	Built into Claude API
Vector DB	Qdrant, Chroma, FAISS	Pinecone, Weaviate Cloud
MCP Servers	Community GitHub servers	Anthropic official servers
Workflow Builder	n8n (self-hosted)	n8n.cloud, Zapier AI
Embedding Model	sentence-transformers	OpenAI ada-002, Cohere

Quick-Start Checklist — Build Your First Agent

- **Step 1:** Get an LLM API key (Anthropic / OpenAI)
 - **Step 2:** Choose a Vector DB (ChromaDB for local, Pinecone for cloud)
 - **Step 3:** Ingest your documents → chunk → embed → store
 - **Step 4:** Set up an MCP server for each external API you need
 - **Step 5:** Open n8n, add Chat Trigger + AI Agent + Chat Model + Memory
 - **Step 6:** Connect tool nodes (web search, your MCP servers)
 - **Step 7:** Write a clear system prompt for your agent
 - **Step 8:** Set Max Iterations to 10–15
 - **Step 9:** Test with a real end-to-end query
 - **Step 10:** Activate workflow and monitor logs
-

